

**Program and Branch Name:**

BTech CSE - F

PBL Progress Report – II

Field	Details
Course Name	Data Structures and Algorithms-II
Course Code	CCSE0301
Faculty Name	Mr. Shamshad Ali
Assignment Type	Individual Assignment
Student(s) Name	Utkarsh Raj Shukla
ERP ID (I Year) / Roll No. (II Year onwards)	2501330100398
Team/Group No. (<i>Group/Capstone only</i>)	N/A
Problem Statement / Project Title	PYQ Topic Frequency Analyser
SDG(s)	SDG 4 – Quality Education
Reporting Month	Month 2
Date of Submission	08/10/2026

Student Declaration

I/We certify that this assignment is my/our own work, with all sources appropriately acknowledged. I/We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s):*Utkarsh Raj Shukla***Date:** 08/10/2026

PBL Progress Report - II

Review 2: Development & Application

Review Coverage: Individual Assignment - Unit 1 and Unit 2 (Trees, Heaps, Graph Traversal, Spanning Trees and Shortest Paths) and Unit 3 (Dynamic Programming).

Field	Details																																				
1. Previous Review Feedback	Knowledge: Okay Presentation: Good Documentation: Okay. Feedback points from the review:																																				
	<table><tr><td>Focus more on DSA-II Unit 1 and Unit 2, and do a deep study of their topics (especially Heap Sort, AVL Tree and BST)</td><td>Done</td></tr><tr><td>Now start the implementation</td><td>Done</td></tr><tr><td>Read more research papers</td><td>In progress</td></tr></table>	Focus more on DSA-II Unit 1 and Unit 2, and do a deep study of their topics (especially Heap Sort, AVL Tree and BST)	Done	Now start the implementation	Done	Read more research papers	In progress																														
	Focus more on DSA-II Unit 1 and Unit 2, and do a deep study of their topics (especially Heap Sort, AVL Tree and BST)	Done																																			
	Now start the implementation	Done																																			
Read more research papers	In progress																																				
2. Action Taken on Previous Feedback	• Deep study of Unit 1 and Unit 2: I revised BST insertion and search, all four AVL rotations (LL, RR, LR, RL), tree traversals, Max-Heap and Heap Sort from Unit 1, and BFS, DFS, connected components, Kruskal, Prim and Dijkstra from Unit 2. • Implementation started and completed for this review: every topic I studied is now implemented as a working module in the web application (see rows 6 and 7). • Research papers: I read the original papers behind the main structures: Adelson-Velsky and Landis (1962) for AVL Trees, Dijkstra (1959) for shortest paths, Kruskal (1956) for MSTs, and Luhn (1958) and Spärck Jones (1972) on using word frequency to measure importance. I am continuing to read more.																																				
3. Research / Data Collection Progress	The main data is previous-year question papers. I continued preparing multi-year PYQ samples and checking how questions are converted into topic names, units, marks and year-wise records, including naming differences such as abbreviations and spelling changes so one topic is not counted twice. For theory I used class notes, lecture PPTs and the research papers above, kept with the project material in GitHub.																																				
4. Unit/Module Concepts Applied	The table below shows the concepts I actually used in the web application. The complexity values are for the operations used by the analyser.																																				
	<table><tr><th>Unit / Concept</th><th>How I used it in the project</th><th>Time</th><th>Space</th></tr><tr><td>Unit 1: AVL Tree</td><td>Stores every topic with its frequency. Each new question searches for the topic and updates it, and rotations keep the tree balanced even when topics arrive in sorted order.</td><td>O(log n)</td><td>O(n)</td></tr><tr><td>Unit 1: Max-Heap / Heap Sort</td><td>Ranks topics by frequency. Extracting from the heap gives the Top-N topics, and Heap Sort gives the full ranked list.</td><td>O(n log n)</td><td>O(n)</td></tr><tr><td>Unit 2: BFS</td><td>Starts from a selected topic and visits directly related topics first, then the next level, to show what usually appears near a topic.</td><td>O(V+E)</td><td>O(V)</td></tr><tr><td>Unit 2: DFS + Components</td><td>Repeated DFS splits the graph into topic clusters that can be revised together.</td><td>O(V+E)</td><td>O(V)</td></tr><tr><td>Unit 2: Kruskal / Prim MST</td><td>Kruskal keeps the strongest links without cycles as a revision backbone. Prim grows from the highest-frequency topic and gives a revision order.</td><td>O(E log E) / O(E log V)</td><td>O(V+E)</td></tr><tr><td>Unit 2: Dijkstra</td><td>Finds the lowest-cost path between two topics. The topics in between are bridge topics.</td><td>O((V+E) log V)</td><td>O(V+E)</td></tr><tr><td>Unit 3: 0/1 Knapsack</td><td>Revision time is the capacity, study hours are weights, frequency or marks are values. Picks the best set of topics for the hours available.</td><td>O(nW)</td><td>O(nW)</td></tr><tr><td>Unit 3: LCS</td><td>Merges close topic-name variants, and compares topic sequences of two papers to see how much repeats.</td><td>O(mn)</td><td>O(mn)</td></tr></table>	Unit / Concept	How I used it in the project	Time	Space	Unit 1: AVL Tree	Stores every topic with its frequency. Each new question searches for the topic and updates it, and rotations keep the tree balanced even when topics arrive in sorted order.	O(log n)	O(n)	Unit 1: Max-Heap / Heap Sort	Ranks topics by frequency. Extracting from the heap gives the Top-N topics, and Heap Sort gives the full ranked list.	O(n log n)	O(n)	Unit 2: BFS	Starts from a selected topic and visits directly related topics first, then the next level, to show what usually appears near a topic.	O(V+E)	O(V)	Unit 2: DFS + Components	Repeated DFS splits the graph into topic clusters that can be revised together.	O(V+E)	O(V)	Unit 2: Kruskal / Prim MST	Kruskal keeps the strongest links without cycles as a revision backbone. Prim grows from the highest-frequency topic and gives a revision order.	O(E log E) / O(E log V)	O(V+E)	Unit 2: Dijkstra	Finds the lowest-cost path between two topics. The topics in between are bridge topics.	O((V+E) log V)	O(V+E)	Unit 3: 0/1 Knapsack	Revision time is the capacity, study hours are weights, frequency or marks are values. Picks the best set of topics for the hours available.	O(nW)	O(nW)	Unit 3: LCS	Merges close topic-name variants, and compares topic sequences of two papers to see how much repeats.	O(mn)	O(mn)
	Unit / Concept	How I used it in the project	Time	Space																																	
	Unit 1: AVL Tree	Stores every topic with its frequency. Each new question searches for the topic and updates it, and rotations keep the tree balanced even when topics arrive in sorted order.	O(log n)	O(n)																																	
	Unit 1: Max-Heap / Heap Sort	Ranks topics by frequency. Extracting from the heap gives the Top-N topics, and Heap Sort gives the full ranked list.	O(n log n)	O(n)																																	
	Unit 2: BFS	Starts from a selected topic and visits directly related topics first, then the next level, to show what usually appears near a topic.	O(V+E)	O(V)																																	
	Unit 2: DFS + Components	Repeated DFS splits the graph into topic clusters that can be revised together.	O(V+E)	O(V)																																	
	Unit 2: Kruskal / Prim MST	Kruskal keeps the strongest links without cycles as a revision backbone. Prim grows from the highest-frequency topic and gives a revision order.	O(E log E) / O(E log V)	O(V+E)																																	
	Unit 2: Dijkstra	Finds the lowest-cost path between two topics. The topics in between are bridge topics.	O((V+E) log V)	O(V+E)																																	
	Unit 3: 0/1 Knapsack	Revision time is the capacity, study hours are weights, frequency or marks are values. Picks the best set of topics for the hours available.	O(nW)	O(nW)																																	
Unit 3: LCS	Merges close topic-name variants, and compares topic sequences of two papers to see how much repeats.	O(mn)	O(mn)																																		
Bellman-Ford, Floyd-Warshall, Transitive Closure, Matrix Chain Multiplication and the Resource Allocation Problem were studied but are not used. Edge costs are positive, so Dijkstra is enough; Knapsack already covers the revision-time selection.																																					
5. Methodology / Approach Followed	Methodology used: Incremental Model. The project was divided into small increments. Each increment adds one data structure or algorithm, and it is built, tested and joined to the working system before the next one starts. How I applied it: • Increment 1 (Month 1, tree layer): file reading and parsing, topic-name normalisation, AVL Tree frequency store and in-order output, then Max-Heap and Heap Sort ranking. • Increment 2 (Month 2, graph and DP layer): weighted co-occurrence graph, BFS and DFS clustering, Kruskal and Prim routes, Dijkstra, and finally 0/1 Knapsack and LCS. • Test before moving on: every increment has its own automated tests (for example, AVL balance on sorted input, and Knapsack checked against a brute-force answer), so a new stage is only built on stages that already work. • Integrate and review: after each increment the full pipeline is run in the web app, and review feedback is applied in the next increment.																																				

PBL Progress Report - II

Review 2: Development & Application (Continued)

Field	Details
6. Work Completed During Month 2	Implementation - Finished the web-based PYQ Topic Frequency Analyser and connected all analysis functions to the web interface, with upload of PDF, Word and PowerPoint papers. - AVL Tree topic store (search, update, balancing, in-order table), Max-Heap Top-N and Heap Sort ranking. - Weighted co-occurrence graph with BFS, DFS and connected components; Kruskal and Prim revision routes; Dijkstra bridge topics. 0/1 Knapsack revision-time allocation and LCS topic-name matching, with automated tests for every module. Deep study of DSA topics Unit 1: BST, AVL Tree and all four rotations, tree traversals, Max-Heap and Heap Sort. Unit 2: graph representation, BFS, DFS, connected components, Kruskal, Prim and Dijkstra, and why Bellman-Ford and Floyd-Warshall are not needed here. Unit 3 concepts: optimal substructure and overlapping subproblems, 0/1 Knapsack and LCS, and Matrix Chain Multiplication and Resource Allocation for comparison.
7. Analysis / Development / Implementation	The code is split into small modules instead of keeping all logic in one file. The parser and extractor prepare question data, the normaliser handles topic-name variants, the AVL Tree stores topic records, the heap module ranks them, the graph module handles traversal, MST and shortest paths, and the DP module contains LCS and Knapsack. The analyser module joins these steps and sends the results to the web interface. The topic graph is stored as an adjacency list. Each vertex is a normalised topic and each edge records how often two topics occur together. The same graph is reused for clustering, revision routes and bridge-topic search. Tests cover AVL balancing and rotations, heap ordering, BFS/DFS and connected components, agreement of Prim and Kruskal MST cost, Dijkstra paths, LCS matching, Knapsack optimal selection and reading of sample question papers.
8. Results / Interim Outcomes	- The app reads PYQ files and produces an alphabetical topic-frequency table and a frequency-based ranked list. - Related topics are grouped into connected clusters, so the result is not a single flat list of high-frequency topics. - Kruskal and Prim produce tree-based routes through a cluster, and Dijkstra shows the bridge topics between two connected topics. - The revision-time option uses 0/1 Knapsack to choose topics within a fixed hour limit instead of taking the first topics from the ranking. - LCS-based matching reduces duplicate counts caused by small differences in topic names.
9. Evidence / Supporting Material	GitHub Repository: https://github.com/utk042/DSA_PBL The repository contains the web app source code, digital notes, lecture PPTs, research paper PDFs, sample PYQ data, program outputs, automated tests and commit history.
10. Challenges & Corrective Actions	Different names for the same topic: wording and abbreviations were splitting the count. I added aliases and LCS-based similarity to merge close variants before storage. Choosing graph edge cost: a raw count works in the opposite direction for shortest paths, so I use 1/count as the cost. Keeping the graph useful: linking every topic from the same paper made the graph too dense, so edges are restricted to topics that share a question or, when selected, a unit. Revision-time values: PYQs have no study-time estimates, so the app uses default hours that the user can change before running Knapsack.
11. Individual / Team Contribution	This is an Individual Assignment. I developed the web application, implemented the data structures and algorithms, connected them to the interface, prepared the sample data and tests, maintained the GitHub repository, and updated the documentation myself.
12. Overall Progress (%)	60%
13. Plan for Final Review	- Read more research papers and add them to the report and repository. - Run the analyser on several real multi-year PYQ sets and check the results against the source papers. - Record runtime observations as the number of questions and topics grows, especially for graph and DP operations. - Prepare the final report and demonstration using the tested version of the web application.

Faculty Evaluation: To be completed by Faculty in Moodle.